

# D-VLA: A High-Concurrency Distributed Asynchronous Reinforcement Learning Framework for Vision-Language-Action Models

Yucheng Guo<sup>5\*</sup>, Yongjian Guo<sup>1,5\*</sup>, Zhong Guan<sup>3,5\*</sup>, Wen Huang<sup>1,5\*</sup>, Haoran Sun<sup>2,5\*</sup>,  
Haodong Yue<sup>1</sup>, Xiaolong Xiang<sup>4,5</sup>, Shuai Di<sup>5</sup>, Zhen Sun<sup>4,5</sup>,  
Luqiao Wang<sup>4,5</sup>, Junwu Xiong<sup>5†</sup>, Yicheng Gong<sup>5</sup>

<sup>1</sup>Tsinghua University, <sup>2</sup>Perking University, <sup>3</sup>Tianjin University, <sup>4</sup>Beihang University, <sup>5</sup>JDT AI Infra

## Abstract

The rapid evolution of Embodied AI has enabled Vision-Language-Action (VLA) models to excel in multimodal perception and task execution. However, applying Reinforcement Learning (RL) to these massive models in large-scale distributed environments faces severe systemic bottlenecks, primarily due to the resource conflict between high-fidelity physical simulation and the intensive VRAM/bandwidth demands of deep learning. This conflict often leaves overall throughput constrained by execution-phase inefficiencies. To address these challenges, we propose *D-VLA*, a high-concurrency, low-latency distributed RL framework for large-scale embodied foundation models. *D-VLA* introduces "Plane Decoupling," physically isolating high-frequency training data from low-frequency weight control to eliminate interference between simulation and optimization. We further design a four-thread asynchronous "Swimlane" pipeline, enabling full parallel overlap of sampling, inference, gradient computation, and parameter distribution. Additionally, a dual-pool VRAM management model and topology-aware replication resolve memory fragmentation and optimize communication efficiency. Experiments on benchmarks like LIBERO show that *D-VLA* significantly outperforms mainstream RL frameworks in throughput and sampling efficiency for billion-parameter VLA models. In trillion-parameter scalability tests, our framework maintains exceptional stability and linear speedup, providing a robust system for high-performance general-purpose embodied agents.

## 1 Introduction

Embodied AI, regarded as a pivotal pathway toward Artificial General Intelligence (AGI), is undergoing a profound paradigm shift driven by the emergence of Vision-Language-Action (VLA) models [1, 2, 3, 4, 5] such as OpenVLA [4],  $\pi_0$  [2], and GR00T [6]. These models achieve a significant transition from manually designed explicit models to data-driven implicit models by integrating visual perception, language understanding, and action generation into a unified end-to-end framework [7, 8, 9].

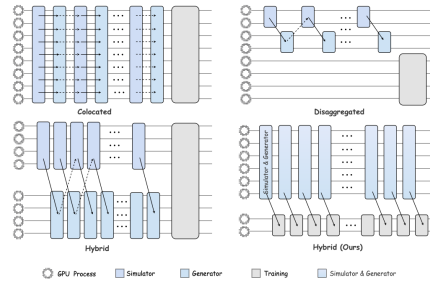


Figure 1: Placement Strategies across Different Training Frameworks

\*Equal contributions.

†Corresponding author. (xiongjunwu.1@jd.com)

Through continuous scaling across massive computational resources and datasets, VLA models have demonstrated unprecedented potential in cross-task and cross-morphology adaptation. However, despite these notable advancements, current training paradigms remain heavily reliant on imitation learning based on Supervised Fine-Tuning (SFT). Existing frameworks like LeRobot [10] and GR00T [6] primarily utilize expert demonstration data to fine-tune policies via behavior cloning. This SFT-centric path faces multiple severe challenges in practical applications: first, large-scale human-collected robot trajectory data is both costly and difficult to obtain, which strictly limits the scaling of models [11, 12]; second, processing high-dimensional data and complex multimodal architectures imposes a heavy burden on training cycles and inference latency. Furthermore, constrained by the limited diversity of offline datasets, SFT models often exhibit weak generalization capabilities when encountering distribution shifts and unseen tasks [13, 14]. Unlike human learning mechanisms, SFT cannot support agents in discovering new action patterns through autonomous exploration beyond demonstration data. Consequently, the research community is increasingly pivoting toward Reinforcement Learning (RL) frameworks to break through these limitations via online interaction.

In response to the deficiencies of SFT, several RL frameworks have emerged, optimizing different dimensions of the training pipeline. For instance, RLinf [15] provides a universal distributed training and evaluation interface, offering a unified framework for multimodal agents and VLAs. RL-VLA<sup>3</sup> [16] implements a three-stage asynchronous pipeline to decouple data collection, policy inference, and model updates, thereby maximizing hardware utilization and training throughput. SimpleVLA-RL [17] designs rule-based outcome rewards and interactive trajectory sampling for VLA models, proving that performance can surpass SFT even with minimal demonstrations. Additionally, Dexbotix [18] focuses on integrating tactile feedback for high-degree-of-freedom dexterous hands, while Vlab [19] is dedicated to building specialized sim-to-real transfer environments.

Unlike traditional online RL, Embodied AI training involves deep coupling between high-fidelity physics simulation and large-scale deep learning models. The former features high-frequency, fragmented occupation of computational resources, while the latter demands extreme throughput for GPU memory capacity and communication bandwidth. Existing distributed training frameworks, such as RLinf-VLA [15] and RL-VLA<sup>3</sup> [16], introduce hybrid resource allocation and fine-grained asynchronous mechanisms to alleviate computational pressure. However, they have yet to fundamentally resolve the resource contention and execution conflicts between simulation tasks and model optimization at the underlying architecture level. As a result, the overall system throughput remains bottlenecked by the slowest physics stepping or synchronization overhead.

The current performance bottlenecks in embodied RL systems primarily stem from the high degree of coupling between simulation logic and learning logic on the execution plane. On one hand, frequent memory allocation and deallocation by physics engines easily lead to severe memory fragmentation in deep learning frameworks. On the other hand, the frequent transfer of massive multimodal environment data (e.g., high-resolution images) between sampling and inference components introduces significant serialization overhead and communication latency. This systemic "blocking" effect is particularly severe when handling long-sequence interaction tasks, restricting the sample acquisition efficiency of agents in complex scenarios.

To address these challenges, we propose *D-VLA*, a high-performance distributed embodied reinforcement learning framework. The core innovation of this framework lies in the design philosophy of "Plane Decoupling," which physically isolates the high-frequency Data Plane from the low-frequency weight Control Plane during the training process, eliminating interference between simulation and training tasks from the ground up. Based on this concept, we have constructed a four-thread asynchronous execution pipeline—the "Swimlane" model. By parallelizing sampling, weight reception, gradient training, and parameter distribution, we achieve full overlap of computation and communication. To further optimize heterogeneous resource utilization, *D-VLA* introduces a dual-pool memory management model and a zero-copy data exchange mechanism, supporting various flexible placement strategies including co-located, separated, and hybrid deployments, as shown in Figure 1. By combining Group Relative Policy Optimization (GRPO) [20] with local topology replication scaling techniques, *D-VLA* successfully breaks through the scalability bottlenecks of large-scale interactive data processing, providing stable support for the training of ultra-large-scale VLA models.

The primary contributions of this paper are summarized as follows:

- **"Plane Decoupling" and Four-Thread Asynchronous Pipeline Architecture:** We propose a system design that physically isolates the high-frequency data interaction plane from the low-frequency weight control plane. Through an innovative four-thread "Swimlane" parallel mechanism, we achieve complete computational overlap of data sampling, policy inference, gradient training, and parameter distribution, resolving resource conflicts between embodied simulation and model optimization at the architectural level.
- **Hierarchical Memory Management and Topology-Aware Scaling Strategies:** we introduce a dual-pool GPU memory management model and a zero-copy data exchange mechanism to effectively alleviate memory fragmentation caused by physics engines. Meanwhile, through local topology replication and control plane offloading techniques, we significantly reduce cross-node communication latency and optimize the communication-to-computation ratio while maintaining global consistency for trillion-parameter models.
- **Performance Breakthrough and Validation in Large-Scale Embodied Tasks:** By integrating the GRPO algorithm into the *D-VLA* framework and conducting extensive validation on complex benchmarks such as LIBERO, we demonstrate that the system possesses superior stability and sampling efficiency when handling long-sequence, large-scale interactive data, significantly outperforming existing mainstream distributed RL baselines.

## 2 Related Work

**Evolution of Vision-Language-Action Models** In the field of Embodied AI, the evolution of Vision-Language-Action (VLA) models is undergoing a paradigm shift from fundamental Supervised Fine-Tuning (SFT) [21] toward Reinforcement Learning (RL) [19] frameworks with enhanced generalization capabilities. Early foundational VLA models such as RT-2 [1], OpenVLA [4], and  $\pi_0$  [2] achieved preliminary robotic manipulation capabilities by training on large-scale datasets like Open-X Embodiment [22]. Subsequently, models like  $\pi_{0.5}$  [23] and SmolVLA [5] further optimized parameter efficiency and inference speed while maintaining performance. Concurrently, general-purpose humanoid control models such as Gr00t N1.5 [6] have demonstrated broad application prospects. However, due to the limitations of SFT methods in handling out-of-distribution data, research focus is gradually shifting toward utilizing RL to enable continuous dynamic environmental adaptation in complex benchmarks such as ManiSkill [24] and LIBERO [25].

**Frameworks for VLA Training and Optimization** To support the efficient training of large-scale VLAs, the academic community has developed a series of optimization frameworks ranging from low-level control to high-level learning. Frameworks such as LeRobot [10] and DexBotix [18] provide vertically integrated toolchains for end-to-end learning, covering the entire process from data processing to policy fine-tuning for algorithms like ACT and Diffusion Policy. Early attempts such as SimpleVLA-RL [17] and RLinf-VLA [15] sought to migrate RLHF workflows from Large Language Models (LLMs) to embodied tasks. Nevertheless, the high latency introduced by physics simulators causes synchronous training pipelines to face severe throughput bottlenecks. To address this, RL-VLA<sup>3</sup> [16] proposed a fully asynchronous distributed architecture that decouples simulation, inference, and training processes. By drawing on the design philosophies of LLM training systems like veRL [11], OpenRLHF [12], and ROLLART [26], it significantly enhances hardware utilization and training efficiency for policy optimization.

**System Challenges in Embodied AI** Embodied AI systems face system-level challenges distinct from those of traditional language models during training, primarily stemming from the uncertainties inherent in physics simulators [21]. Unlike the relatively stable neural reward models used in LLM training [27], VLA training requires frequent calls to simulation environments like RoboCasa [28]. These environments exhibit high and fluctuating CPU and GPU resource consumption, which can easily lead to computational idling and resource waste [24, 28]. To address these pain points, frontier research such as RL-VLA<sup>3</sup> [16] has introduced dynamic batch scheduling and fine-grained environment sharding techniques, aiming to resolve the asynchronous bottlenecks caused by simulators while maximizing both sample efficiency and system throughput.



flow at the memory level and reducing bandwidth consumption. For tasks requiring massive parallel simulation, *D-VLA* utilizes a separated deployment strategy with pipeline communication, allowing the environment side to perform physical stepping while the inference side concurrently processes the previous decision logic.

### 3.3 Decoupled Communication Architecture

A key innovation of the *D-VLA* architecture is its physical plane decoupling technology, which distinguishes the “Training Data Plane” from the “Weight Control Plane”. In embodied AI training, the environment interaction data generated during the rollout phase constitutes the primary load of the Data Plane due to its high update frequency and volume. We utilize high-performance collective communication libraries to dynamically switch communication paradigms based on resource ratios. In symmetric deployments, efficient point-to-point communication is used; in asymmetric deployments, dynamic networking enables global data exchange and metadata synchronization via master-node broadcasting.

Concurrently, model weight synchronization is assigned to the Weight Control Plane. Unlike the Data Plane, weight distribution is less frequent but requires high determinism. *D-VLA* offloads this logic to a CPU-based communication backend, using host-side contiguous buffers for broadcasting. This design avoids the GPU synchronization calls common in physical simulation paths, which often cause stream contention or deadlocks in existing frameworks. Through this dual-plane isolation, the Data Plane focuses on peak throughput while the Control Plane maintains global model consistency

### 3.4 End-to-End Asynchronous Pipeline and Data Flow

*D-VLA* implements a fully non-blocking end-to-end data flow throughout its execution cycle. Environmental features collected by Rollout components are pushed to Actor components in real-time. To mitigate the speed mismatch between data production and training, a resource buffer queue based on host memory is constructed on the Actor side, ensuring continuous sampling without blocking the simulators. At the algorithmic level, *D-VLA* integrates Group Relative Policy Optimization (GRPO) combined with micro-batch training, which captures sparse reward signals in embodied tasks and fits perfectly with the asynchronous architecture.

We define this four-thread architecture as the “Swimlane” model. The main sampling thread, asynchronous weight receiving thread, training execution thread, and weight distribution thread run on their own physical resource tracks, synchronized via lightweight semaphores. This ensures that hardware never idles while waiting for specific signals. Compared to traditional synchronous distributed RL, this design elevates hardware utilization by hiding communication within the computation process.

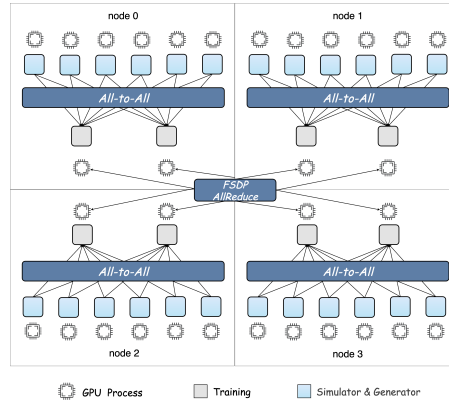


Figure 3: Schematic of Multi-Node Communication in *D-VLA*

### 3.5 Memory Management and Large-Scale Node Scaling

To prevent memory fragmentation caused by non-framework components like physics engines, *D-VLA* proposes a dual-pool memory management model. Memory is explicitly partitioned into a “Model Computation Pool” (managed by Torch’s caching allocator for weights and gradients) and an “Environment Auxiliary Pool” (reserved for physics engine temporary objects like contact points) as shown in Figure 2. This physical isolation prevents framework memory crashes during frequent allocation and deallocation by simulation components.

For cross-node scaling, *D-VLA* employs a local topology replication strategy, as shown in Figure 3. Recognizing that most communication occurs between sampling and inference, *D-VLA* builds a

complete sampling-inference closed loop within a node and replicates this topology as a basic unit across the cluster. This limits high-frequency tensor flows to local high-speed interconnects. For global gradient reduction in large clusters, *D-VLA* combines Fully Sharded Data Parallel (FSDP) technology. Since weight broadcasting is offloaded to the control plane, global synchronization does not negatively impact local sampling efficiency, optimizing the communication-to-computation ratio and increasing overall throughput.

## 4 Experiments

In this section, we conduct a comprehensive empirical evaluation of *D-VLA* against several state-of-the-art (SOTA) orchestration frameworks. Our analysis focuses on computational efficiency, throughput scalability, and hardware utilization in the context of Vision-Language-Action (VLA) training. The results demonstrate that *D-VLA* significantly optimizes the training pipeline across diverse model architectures and resource configurations, effectively mitigating the hardware underutilization commonly encountered in heterogeneous embodied AI workloads.

### 4.1 Experimental Setup

**Model Architectures.** To evaluate the generalizability of *D-VLA*, we select two representative VLA paradigms:  $\pi_{0.5}$ , a diffusion-based model utilizing iterative denoising processes, and OpenVLA-OFT, an auto-regressive Transformer-based model employing Parameter-Efficient Fine-Tuning (PEFT). Both models are configured for action chunking prediction—predicting a sequence of future actions rather than single-step control signals—to enhance environment interaction rates and maximize system efficiency in high-frequency control tasks.

**Simulation Environment.** All experiments are conducted within the ManiSkill physical simulation framework. Unlike traditional environments that rely on CPU-bound physics (e.g., Gym or MuJoCo), ManiSkill leverages high-concurrency GPU rendering and parallelized physics kernels for agent-environment interactions. This introduces significant heterogeneous resource contention, as the simulation process and VLA model inference must simultaneously compete for GPU memory and compute cycles, posing a substantial challenge to the underlying orchestration system.

**Baselines and Placement Strategies.** We benchmark our framework against two state-of-the-art orchestration frameworks under their representative resource configurations on an 16-GPU cluster:

RLinf-VLA, a versatile baseline supporting three deployment modes with its standard protocols: *colocated* (RLinf-co), where all components share the GPU pool; *disaggregated* (RLinf-dis), which allocates 2 GPUs for Rollout/Environment and 4 GPUs for the Actor; and *hybrid* (RLinf-hyper), where the Actor utilizes all GPUs during the training phase. RL-VLA<sup>3</sup>, a fully asynchronous framework that adopts a three-stage "Environment-Rollout-Actor" pipeline. Its placement follows a disaggregated design similar to RLinf-dis, segregating Environment/Rollout and Actor onto 2 and 4 GPUs, respectively. *D-VLA* utilizes a Hybrid Asynchronous Orchestration strategy: 4 GPUs are shared by Rollout and Environment components to minimize data transfer overhead via locality, while the Actor components—responsible for heavy gradient updates and large-model inference—are isolated on a dedicated 4-GPU group. This architectural isolation is a core design principle of *D-VLA*, aimed at preventing kernel-level computational interference and maximizing hardware occupancy.

**Metrics:** To evaluate system efficiency, we utilize **throughput** as the core metric, defined as the total number of environment state transitions processed per unit of time. Given a constant *action chunk size*, this is mathematically equivalent to the total number of action inference steps executed by the rollout generators per second. To ensure the reliability of the experimental results, we maintain identical configurations across all evaluated methods, specifically the rollout batch size and the actor training batch size.

**System Throughput Enhancement.** As shown in Figure 4 and Figure 5,  $\pi_{0.5}$  and OpenVLA-OFT exhibit distinct throughput characteristics due to their inherent computational densities. The traditional RLinf-co (colocated) scheme, while providing a stable baseline, suffers from severe resource utilization bottlenecks due to its inherent "lock-step" synchronization mode. This leads to significant "GPU bubbles" during high-concurrency environment sampling.

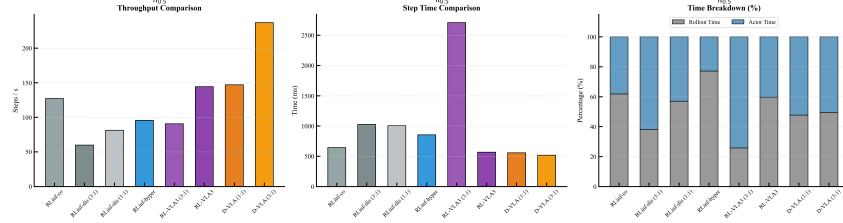


Figure 4: Performance Benchmarking of  $\pi_{0.5}$  under Different Distributed Strategies. (Left) System throughput measured in steps per second; (Middle) Average inference latency per step in milliseconds; (Right) Percentage breakdown of execution time between Rollout and Actor components. Ratios (3:1 and 1:1) represent the resource partitioning between rollout/environment and actor modules.

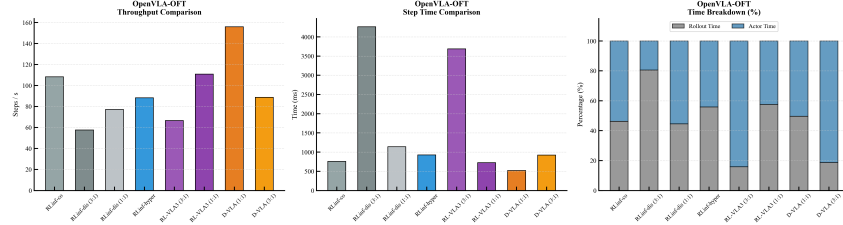


Figure 5: Performance Evaluation of OpenVLA-OFT across Various Scaling Configurations. (Left) Comparison of throughput efficiency; (Middle) Comparison of single-step processing time; (Right) Proportional distribution of time consumption for Rollout and Actor processes.

*D-VLA* achieves substantial performance breakthroughs across both model architectures. In  $\pi_{0.5}$  experiments, the 1:1 configuration reaches a throughput of 147.0 steps/s, a 22.25% improvement over RLinf-co (127.24 steps/s). By optimizing the resource ratio to 3:1, the throughput surges to 237.0 steps/s, representing an 86.26% increase over the baseline. For the more parameter-heavy OpenVLA-OFT, *D-VLA* consistently outperforms the competition, achieving 156.0 steps/s—surpassing RLinf-co (108.24 steps/s) and RL-VLA<sup>3</sup> (110.88 steps/s) by 44.44%. This improvement confirms that our asynchronous strategy, combined with communication optimization, effectively mitigates GPU idling during heterogeneous task switching.

**Pipeline Latency and Asynchronous Overlapping.** Figures 2 and 3 provide a decoupled comparison of Step Time, Rollout Time, and Actor Time to clarify the optimization mechanisms. *D-VLA* demonstrates superior latency control: in  $\pi_{0.5}$  tasks, the total step time is only 566.41 s, a 50.43% reduction compared to RLinf-dis (1006.8 s). Even for OpenVLA-OFT, which possesses high inherent inference latency, *D-VLA* restricts the total time to 520.3 s, significantly outperforming RLinf-hyper. This proves that efficient scheduling can effectively "mask" the computational burden of large-model inference by overlapping it with the subsequent environment sampling phase through refined pipeline design.

**Bottleneck Analysis and Resource Balancing.** Decoupled timing in Figure 4 reveals the dynamic shift of system bottlenecks. In the  $\pi_{0.5}$  3:1 configuration, the Rollout and Actor times are relatively balanced. This workload symmetry allows the asynchronous pipeline to achieve near-perfect mutual masking, maximizing overall throughput.

However, the OpenVLA-OFT experiment highlights the negative impact of resource imbalance. In the 3:1 configuration, the Actor time reaches 542.12 s due to the heavy inference load, becoming the primary bottleneck. This forced wait causes the system to degenerate into a quasi-synchronous mode. To address this, *D-VLA* utilizes an adaptive resource adjustment (1:1 ratio) to align both components at approximately 200 s. This re-establishes pipeline symmetry and restores mutual masking efficiency. Our analysis indicates that the efficacy of asynchronous mechanisms depends heavily on temporal alignment between components. By precisely compressing GPU bubbles and eliminating idle waiting, *D-VLA* ensures optimal throughput across varying VLA model scales.

**Multi-node Scalability.** To verify the robustness of our framework in large-scale settings, we extend our evaluation to a 16-GPU multi-node environment (Table 1). The experimental results are



consistent with our single-node findings, confirming that by leveraging the underlying network fabric (e.g., InfiniBand) for asynchronous weight and data transfers, *D-VLA* achieves efficient scaling in large-scale distributed scenarios without being bottlenecked by inter-node communication.

Table 1: Comparison of throughput performance across different methods for  $\pi_{0.5}$  and OpenVLA-OFT within the ManiSkill simulation environment using 16 GPUs. Here, Thr, Step, Roll, and Act denote Throughput, Step Time, Rollout Time, and Actor Time, respectively. Ratios (3:1 and 1:1) represent the resource partitioning between rollout/environment and actor modules.

| Framework                 | $\pi_{0.5}$ Models |         |         |         | OpenVLA-OFT Models |         |        |         |
|---------------------------|--------------------|---------|---------|---------|--------------------|---------|--------|---------|
|                           | Thr. $\uparrow$    | Step    | Roll.   | Act.    | $\uparrow$ Thr.    | Step    | Roll.  | Act.    |
| RLinf-co                  | 232.23             | 705.50  | 503.50  | 200.50  | 87.20              | 1877.00 | 259.00 | 1607.50 |
| RLinf-dis (1:1)           | 150.58             | 1088.00 | 672.60  | 383.60  | 107.23             | 1528.00 | 508.52 | 1001.80 |
| RLinf-dis (3:1)           | 175.29             | 2840.00 | 1662.02 | 1142.30 | 99.33              | 4948.00 | 896.60 | 4030.23 |
| RLinf-hyper               | 171.74             | 954.00  | 751.20  | 199.00  | 77.23              | 2123.50 | 432.75 | 1675.00 |
| RL-VLA <sup>3</sup> (1:1) | 244.61             | 669.80  | 525.66  | 383.00  | 170.48             | 961.03  | 354.20 | 814.40  |
| RL-VLA <sup>3</sup> (3:1) | 250.77             | 1960.00 | 1580.24 | 1420.33 | 152.98             | 1058.35 | 761.60 | 816.40  |
| <i>D-VLA</i> (1:1)        | <b>336.04</b>      | 488.32  | 460.65  | 452.33  | <b>250.90</b>      | 653.23  | 508.23 | 568.24  |
| <i>D-VLA</i> (3:1)        | <b>376.00</b>      | 1307.23 | 1299.20 | 1288.02 | <b>154.23</b>      | 1062.30 | 708.23 | 868.24  |

**Learning Performance and Convergence.** The success rate curves, as illustrated in Figure 6, demonstrate that while *D-VLA* significantly accelerates the training process, it maintains competitive performance levels consistent with existing baselines. This confirms that our asynchronous orchestration and plane-decoupling mechanisms do not compromise the training stability or the final policy quality of VLA models.

## 4.2 Scalability Analysis and Bottleneck Exploration

To further investigate the capacity and performance evolution of *D-VLA* under large-scale parallel workloads, we conduct a systematic evaluation using the  $\pi_{0.5}$  model as a benchmark with a 3:1 resource placement strategy. We scale the environment count from 384 to 3,072 and monitor the dynamic changes in system throughput and sub-component latencies. The results demonstrate that while the asynchronous pipeline mechanism yields significant performance gains under high-concurrency loads, it also reveals performance saturation points dictated by hardware topology and computational density.

**Throughput Evolution and Saturation Analysis.** As illustrated in Figure 7 (left), the system throughput exhibits a non-linear progression—initially climbing rapidly, then plateauing, and eventually showing a slight decline as the environment count increases. As the environment scale expands from 384 to 768, throughput achieves a significant leap, reaching a peak performance of 379 steps/s, at the 768 scale. This phenomenon reflects the ability of our asynchronous framework to efficiently release GPU parallel potential by masking initial computational bubbles. However, as the scale further expands to 3,072, throughput gradually recedes and stabilizes around 360 steps/s. This performance degradation is not caused by scheduling inefficiencies but by the saturation of GPU memory bandwidth and compute units under high-concurrency rendering, where the increased latency per environment instance eventually offsets the benefits of higher parallelism. This defines the optimal environmental workload range for  $\pi_{0.5}$  training under current hardware configurations.

**Component Decoupling and Pipeline Efficiency.** The linear evolution analysis of Total Step Time, Actor Time, and Rollout Time in Figure 7 (middle) reveals varying sensitivities to system load across different components. As the environment count increases, both Rollout and Actor times exhibit highly stable linear growth, confirming the predictability and rigor of the *D-VLA* scheduling strategy. Crucially, although both Actor and Rollout times scale with the load, their magnitudes remain relatively balanced across all tested scales. According to asynchronous pipeline theory, this temporal symmetry between components is a prerequisite for achieving efficient masking. The

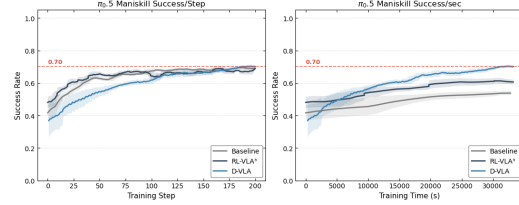


Figure 6: Training Success Rate on ManiSkill with  $\pi_{0.5}$ .



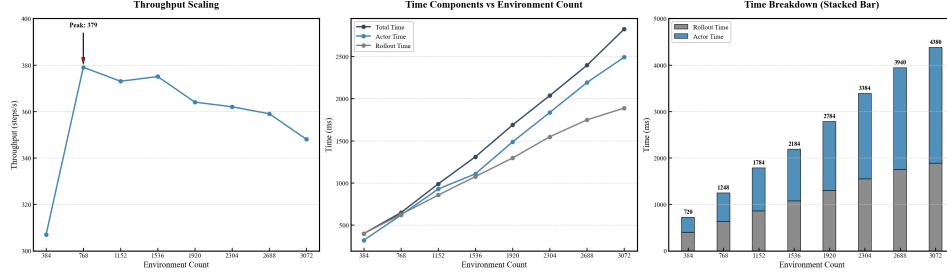


Figure 7: Performance scaling of  $D-VLA$  on  $\pi_{0.5}$  across varying environment counts. The plots illustrate (Left) throughput scaling trends with a peak at 768 environments, (Middle) the linear growth of decoupled time components, and (Right) the stacked breakdown of Rollout and Actor latencies.

experimental data confirms that through precise pipeline alignment, our framework ensures a high degree of overlap between large-model inference and large-scale environment simulation, maintaining a superior system duty cycle even as total step time increases with load.

**Workload Backlog and Bottleneck Shifting.** The stacked latency analysis in Figure 4 (right) further reveals the shifting of system bottlenecks under heavy workloads. At low environment counts (e.g., 384), the absolute latencies of Rollout and Actor are short, and the system possesses redundant compute capacity; during this phase, throughput is primarily constrained by the rollout process. As the environment count increases to 768, the Actor and Rollout times converge, allowing for optimal mutual masking and thus achieving peak throughput. When the load exceeds 1,536, the growth rate of Actor time slightly outpaces that of the Rollout, gradually evolving into the core factor limiting further throughput breakthroughs. This shift reflects the extreme pressure that the complexity of diffusion model computational graphs exerts on compute units when handling massive concurrent inference requests. While the asynchronous masking mechanism significantly compresses the absolute span of a single step cycle, it cannot eliminate the physical latency growth dictated by hardware limits. The scalability performance of  $\pi_{0.5}$  on  $D-VLA$  demonstrates the framework’s academic depth and engineering feasibility for supporting ultra-large-scale parallel training, while highlighting the importance of identifying the "compute-latency" equilibrium by dynamically adjusting environment scales in heterogeneous pipeline designs.

## 5 Conclusion and Discussion

**Conclusion.** This paper presents  $D-VLA$ , a high-performance distributed RL framework for VLA training. By introducing “Plane Decoupling,”  $D-VLA$  physically isolates high-frequency data exchange from low-frequency weight control, resolving resource contention between simulation and optimization. Our four-thread “Swimlane” pipeline enables full overlap of sampling, inference, and training, reducing GPU idle time. Experiments on  $\pi_{0.5}$  and OpenVLA-OFT show up to 86% throughput gains over SOTA baselines without compromising convergence or policy quality.

**Discussion.** Analysis reveals that VLA training efficiency depends on temporal alignment between components. While  $D-VLA$  mitigates synchronization bottlenecks, scaling to trillion-parameter models requires even finer pipeline symmetry. Future work will investigate dynamic, load-aware resource reallocation to adaptively adjust GPU partitioning based on real-time latencies. We also plan to extend  $D-VLA$  to multi-agent scenarios and more heterogeneous embodied platforms to further scale generalist foundation models.

## References

- [1] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander

- Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 06–09 Nov 2023.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al.  $\pi_0$ : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [3] Gemini Robotics Team. Gemini Robotics 1.5: Pushing the Frontier of Generalist Robots with Advanced Embodied Reasoning, Thinking, and Motion Transfer. *arXiv e-prints*, page arXiv:2510.03342, October 2025.
- [4] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [5] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025.
- [6] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [7] Yuen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. A survey on vision-language-action models for embodied ai. *arXiv preprint arXiv:2405.14093*, 2024.
- [8] Yifan Zhong, Fengshuo Bai, Shaofei Cai, Xuchuan Huang, Zhang Chen, Xiaowei Zhang, Yuanfei Wang, Shaoyang Guo, Tianrui Guan, Ka Nam Lui, et al. A survey on vision-language-action models: An action tokenization perspective. *arXiv preprint arXiv:2507.01925*, 2025.
- [9] Dapeng Zhang, Jing Sun, Chenghui Hu, Xiaoyan Wu, Zhenlong Yuan, Rui Zhou, Fei Shen, and Qingguo Zhou. Pure vision language action (vla) models: A comprehensive survey. *arXiv preprint arXiv:2509.19012*, 2025.
- [10] Remi Cadene, Simon Aliberts, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Jade Choghari, Martino Russi, Caroline Pascal, Steven Palma, et al. Lerobot: An open-source library for end-to-end robot learning. *arXiv preprint arXiv:2602.22818*, 2026.
- [11] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.
- [12] Jian Hu, Xibin Wu, Zilin Zhu, Weixun Wang, Dehao Zhang, Yu Cao, et al. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 6, 2024.
- [13] Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.
- [14] Zhenpeng Su, Leiyu Pan, Xue Bai, Dening Liu, Guanting Dong, Jiaming Huang, Minxuan Lv, Wenping Hu, Fuzheng Zhang, Kun Gai, et al. Klear-reasoner: Advancing reasoning capability via gradient-preserving clipping policy optimization. *arXiv preprint arXiv:2508.07629*, 2025.
- [15] Hongzhi Zang, Mingjie Wei, Si Xu, Yongji Wu, Zhen Guo, Yuanqing Wang, Hao Lin, Liangzhi Shi, Yuqing Xie, Zhexuan Xu, et al. Rlinf-vla: A unified and efficient framework for vla+ rl training. *arXiv preprint arXiv:2510.06710*, 2025.

- [16] Zhong Guan, Haoran Sun, Yongjian Guo, Shuai Di, Xiaodong Bai, Jing Long, Tianyun Zhao, Mingxi Luo, Chen Zhou, Yucheng Guo, et al. RL-vla3: Reinforcement learning vla accelerating via full asynchronism. *arXiv preprint arXiv:2602.05765*, 2026.
- [17] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. Simplevla-rl: Scaling vla training via reinforcement learning. *arXiv preprint arXiv:2509.09674*, 2025.
- [18] Bin Xie, Erjin Zhou, Fan Jia, Hao Shi, Haoqiang Fan, Haowei Zhang, Hebei Li, Jianjian Sun, Jie Bin, Junwen Huang, et al. Dexbotic: Open-source vision-language-action toolbox. *arXiv preprint arXiv:2510.23511*, 2025.
- [19] Mustafa Shukor Dana Aubakirova, Jade Cholgari, and Leandro von Werra. Vlab: Your laboratory for pretraining vlas. <https://github.com/huggingface/vlab>, 2025.
- [20] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [21] Chen Zhou, Haoran Sun, Hedan Yang, Jing Long, Junwu Xiong, Luqiao Wang, Mingxi Luo, Qiming Yang, Shuai Di, Song Wang, et al. Thousand-gpu large-scale training and optimization recipe for ai-native cloud embodied intelligence infrastructure. *arXiv preprint arXiv:2603.11101*, 2026.
- [22] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.
- [23] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. pi 0.5: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [24] T Mu, Z Ling, F Xiang, D Yang, X Li, S Tao, Z Huang, Z Jia, and H Su. Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations. In *35th Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.
- [25] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [26] Wei Gao, Yuheng Zhao, Tianyuan Wu, Shaopan Xiong, Weixun Wang, Dakai An, Lunxi Cao, Dilxat Muhtar, Zichen Liu, Haizhou Zhao, et al. Rollart: Scaling agentic rl training via disaggregated infrastructure. *arXiv preprint arXiv:2512.22560*, 2025.
- [27] Rui Yu, Shenghua Wan, Yucen Wang, Chen-Xiao Gao, Le Gan, Zongzhang Zhang, and De-Chuan Zhan. Reward models in deep reinforcement learning: A survey. *arXiv preprint arXiv:2506.15421*, 2025.
- [28] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. *arXiv preprint arXiv:2406.02523*, 2024.